

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os
import shutil
from sklearn.model_selection import train_test_split
from google.colab import drive
# Mount Google Drive
drive.mount('/content/drive')
# Define paths
dataset_folder = "/content/drive/MyDrive/Top View Pics" # Original dataset folder
output_train_folder = "/content/drive/My Drive/output/train"
output_test_folder = "/content/drive/My Drive/output/test"
# Ensure output directories exist
os.makedirs(output_train_folder, exist_ok=True)
os.makedirs(output_test_folder, exist_ok=True)
# Define labels based on subfolder names
labels = ['AD Top View', 'CN Top View', 'MCI Top View']
# Check and process each category folder
for label in labels:
    section_path = os.path.join(dataset_folder, label)
    # Skip if not a directory
    if not os.path.isdir(section_path):
        print(f"Directory not found for label: {label}. Skipping...")
        continue
    # List all files in the section folder
    files = [f for f in os.listdir(section_path) if os.path.isfile(os.path.join(section_path, f))]
    if not files:
        print(f"No files found in section: {label}. Skipping...")
        continue
    # Split files into training and testing sets
    train_files, test_files = train_test_split(files, test_size=0.2, random_state=42)
    # Create output directories for the current section
    train_section_path = os.path.join(output_train_folder, label)
    test_section_path = os.path.join(output_test_folder, label)
    os.makedirs(train_section_path, exist_ok=True)
    os.makedirs(test_section_path, exist_ok=True)
    # Copy training files
    for file in train_files:
        src = os.path.join(section_path, file)
        dst = os.path.join(train_section_path, file)
        shutil.copy(src, dst)
    # Copy testing files
    for file in test_files:
        src = os.path.join(section_path, file)
        dst = os.path.join(test_section_path, file)
        shutil.copy(src, dst)
    # Print summary
    total_files = len(files)
    train_count = len(train_files)
    test_count = len(test_files)
    print(f"Section: {label}")
    print(f"Total files: {total_files}")
    print(f"Training files: {train_count} ({(train_count / total_files) * 100:.2f}%)")
    print(f"Testing files: {test_count} ({(test_count / total_files) * 100:.2f}%)")
    print("-" * 50)
print("Dataset successfully split into training and testing sets!")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
Section: AD Top View
Total files: 636
Training files: 508 (79.87%)
Testing files: 128 (20.13%)
-----
Section: CN Top View
Total files: 1344
Training files: 1075 (79.99%)
Testing files: 269 (20.01%)
-----
Section: MCI Top View
Total files: 1440
Training files: 1152 (80.00%)
Testing files: 288 (20.00%)
-----
```

What can I help you build?

Dataset successfully split into training and testing sets!

```
import cv2
import numpy as np
from tqdm import tqdm
from sklearn.utils import shuffle
from tensorflow.keras.utils import to_categorical
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB7
from tensorflow.keras.metrics import CategoricalAccuracy
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Paths for train and test folders
train_path = "/content/drive/MyDrive/output/train"
test_path = "/content/drive/MyDrive/output/test"
# Labels corresponding to folder names
labels = ['AD Top View', 'CN Top View', 'MCI Top View']
# Image size for resizing
image_size = 128 # Reduced to 128x128 for memory optimization
# Helper function to load and preprocess images
def load_images(folder_path, label, data_list, label_list):
    for image_file in tqdm(os.listdir(folder_path), desc=f>Loading {label}"):
        image_path = os.path.join(folder_path, image_file)
        image = cv2.imread(image_path) # Load the image
        if image is not None:
            image = cv2.resize(image, (image_size, image_size)) # Resize to 128x128
            image = image / 255.0 # Normalize pixel values
            data_list.append(image)
            label_list.append(label)
# Initialize lists for training and testing data
X_train, Y_train = [], []
X_test, Y_test = [], []
# Load training data
for label in labels:
    folder_path = os.path.join(train_path, label)
    if os.path.exists(folder_path):
        load_images(folder_path, label, X_train, Y_train)
    else:
        print(f"Training folder for {label} not found. Skipping...")
# Load testing data
for label in labels:
    folder_path = os.path.join(test_path, label)
    if os.path.exists(folder_path):
        load_images(folder_path, label, X_test, Y_test)
    else:
        print(f"Testing folder for {label} not found. Skipping...")
# Convert to NumPy arrays with reduced memory usage
X_train = np.array(X_train, dtype=np.float32)
Y_train = np.array(Y_train)
X_test = np.array(X_test, dtype=np.float32)
Y_test = np.array(Y_test)
# Shuffle training data
X_train, Y_train = shuffle(X_train, Y_train, random_state=42)
# Convert labels from strings to integers
label_to_int = {label: idx for idx, label in enumerate(labels)}
Y_train_int = np.array([label_to_int[label] for label in Y_train])
Y_test_int = np.array([label_to_int[label] for label in Y_test])
# One-hot encode the labels
Y_train_onehot = to_categorical(Y_train_int, num_classes=len(labels))
Y_test_onehot = to_categorical(Y_test_int, num_classes=len(labels))
# Print shapes for verification
print("Training data shape:", X_train.shape)
print("Training labels shape:", Y_train_onehot.shape)
print("Testing data shape:", X_test.shape)
print("Testing labels shape:", Y_test_onehot.shape)
```

```
↻ Loading AD Top View: 100%|██████████| 508/508 [00:04<00:00, 111.20it/s]
Loading CN Top View: 100%|██████████| 1075/1075 [00:12<00:00, 85.75it/s]
Loading MCI Top View: 100%|██████████| 1152/1152 [00:13<00:00, 83.90it/s]
Loading AD Top View: 100%|██████████| 128/128 [00:01<00:00, 124.25it/s]
Loading CN Top View: 100%|██████████| 269/269 [00:02<00:00, 115.94it/s]
Loading MCI Top View: 100%|██████████| 288/288 [00:02<00:00, 107.00it/s]
Training data shape: (2735, 128, 128, 3)
Training labels shape: (2735, 3)
Testing data shape: (685, 128, 128, 3)
```

Testing labels shape: (685, 3)

```
# Define the EfficientNetB7 model
base_model = EfficientNetB7(weights='imagenet', include_top=False, input_shape=(image_size, image_size, 3))
base_model.trainable = False # Freeze base model layers to reduce memory usage
# Add custom layers
x = base_model.output
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.5)(x)
x = tf.keras.layers.Dense(3, activation='softmax')(x) # 3 classes: AD, CN, MCI
# Create final model
model = tf.keras.models.Model(inputs=base_model.input, outputs=x)
# Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=[CategoricalAccuracy(name='accuracy')]
)
# Train the model with reduced batch size
history = model.fit(
    X_train, Y_train_onehot,
    validation_data=(X_test, Y_test_onehot),
    epochs=10,
    batch_size=16, # Reduced batch size to fit in 12GB RAM
    verbose=1
)
# Evaluate the model
test_loss, test_accuracy = model.evaluate(X_test, Y_test_onehot, verbose=2)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
# Predictions on the test set
pred = model.predict(X_test)
pred = np.argmax(pred, axis=1)
actual_label = np.argmax(Y_test_onehot, axis=1)
# Classification report
print(classification_report(actual_label, pred, target_names=labels))
# Confusion matrix
cm = confusion_matrix(actual_label, pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, cmap='Blues', annot=True, fmt='d', xticklabels=labels, yticklabels=labels)
plt.title("Confusion Matrix for Alzheimer's Disease Classification (EfficientNetB7)")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.savefig('efficientnetb7_confusion.png', dpi=300)
plt.show()
```

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb7_notop.h5
 258076736/258076736 — 2s 0us/step

Epoch 1/10
 171/171 — 618s 3s/step - accuracy: 0.3703 - loss: 1.3719 - val_accuracy: 0.4204 - val_loss: 1.0672

Epoch 2/10
 171/171 — 645s 3s/step - accuracy: 0.3933 - loss: 1.2602 - val_accuracy: 0.4204 - val_loss: 1.0824

Epoch 3/10
 171/171 — 613s 3s/step - accuracy: 0.4077 - loss: 1.2120 - val_accuracy: 0.3927 - val_loss: 1.1019

Epoch 4/10
 171/171 — 623s 3s/step - accuracy: 0.4058 - loss: 1.2471 - val_accuracy: 0.3927 - val_loss: 1.0657

Epoch 5/10
 171/171 — 588s 3s/step - accuracy: 0.3969 - loss: 1.2042 - val_accuracy: 0.3927 - val_loss: 1.1942

Epoch 6/10
 171/171 — 561s 3s/step - accuracy: 0.3905 - loss: 1.2111 - val_accuracy: 0.3927 - val_loss: 1.1813

Epoch 7/10
 171/171 — 594s 3s/step - accuracy: 0.3969 - loss: 1.2507 - val_accuracy: 0.4204 - val_loss: 1.4182

Epoch 8/10
 171/171 — 622s 3s/step - accuracy: 0.3880 - loss: 1.2609 - val_accuracy: 0.1869 - val_loss: 1.1068

Epoch 9/10
 171/171 — 623s 3s/step - accuracy: 0.3844 - loss: 1.1662 - val_accuracy: 0.3927 - val_loss: 1.0712

Epoch 10/10
 171/171 — 622s 3s/step - accuracy: 0.3757 - loss: 1.1662 - val_accuracy: 0.4204 - val_loss: 1.2202

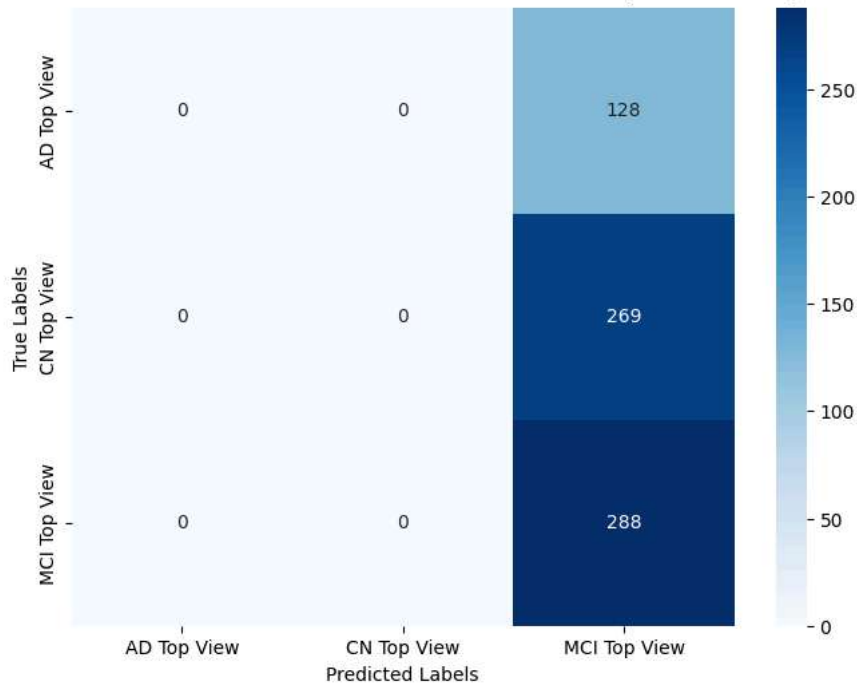
22/22 - 118s - 5s/step - accuracy: 0.4204 - loss: 1.2202
 Test Loss: 1.2202
 Test Accuracy: 42.04%

22/22 — 138s 6s/step

```
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

	precision	recall	f1-score	support
AD Top View	0.00	0.00	0.00	128
CN Top View	0.00	0.00	0.00	269
MCI Top View	0.42	1.00	0.59	288
accuracy			0.42	685
macro avg	0.14	0.33	0.20	685
weighted avg	0.18	0.42	0.25	685

Confusion Matrix for Alzheimer's Disease Classification (EfficientNetB7)



```
# =====
# ♦ Plot Training & Validation Accuracy
# =====
plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy', marker='o')
plt.title('Model Accuracy')
```

```

plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# =====
# ♦ Plot Training & Validation Loss
# =====
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Train Loss', marker='o')
plt.plot(history.history['val_loss'], label='Validation Loss', marker='o')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()

```

